CHAPTER - 1

INTRODUCTION

The Anviksha project is an educational initiative focused on the design, simulation, and implementation of an 8-bit Central Processing Unit (CPU)**.** The project aims to provide students with a practical understanding of computer architecture by demonstrating how a CPU executes instructions, processes data, and controls overall system operations. By combining theoretical concepts with digital design and hardware implementation, the project bridges the gap between classroom learning and practical application, enabling learners to gain a deeper understanding of processor architecture.

At the core of the project is an 8-bit CPU consisting of essential components such as the Arithmetic Logic Unit (ALU), registers, program counter, instruction register, memory unit, and control unit**.** These components work together to perform the instruction cycle, including fetch, decode, and execute operations. Compared to a basic 4-bit processor, the 8-bit architecture provides enhanced data processing capability while remaining simple enough for educational purposes. This allows students to understand the interaction between different processor components and the flow of data during instruction execution without the complexity of modern commercial processors.

The project utilizes Logisim Evolution for the schematic design and simulation of the CPU. Logisim provides an interactive environment where individual modules can be designed, tested, and verified before integration into the complete processor. Through simulation, students can visualize the operation of each hardware component, analyze data flow, and validate the correctness of instruction execution. This stage helps in identifying design issues at an early stage and ensures reliable processor functionality before hardware implementation.

Following successful simulation, the CPU is implemented on an FPGA (Field-Programmable Gate Array) using Verilog**,** a widely used Hardware Description Language (HDL). FPGA implementation enables the simulated design to be realized as a working hardware system, providing practical experience in digital circuit design and hardware development. This process familiarizes students with FPGA design flow, hardware verification, and real-world implementation techniques that are widely adopted in embedded systems and VLSI design.

Overall, the Anviksha 8-bit CPU project serves as an effective learning platform for understanding the fundamental principles of processor design, digital logic, and computer organization. By integrating simulation, hardware implementation, and modular design, the project strengthens both theoretical knowledge and practical skills. It also provides a strong foundation for further study in embedded systems, computer architecture, FPGA-based system design, and VLSI engineering.

1.1 Objectives of the Study

The Anviksha – 8-bit CPU project is driven by several key objectives, each aimed at enhancing students' understanding of processor architecture, digital logic design, and hardware implementation.

1.1.1 Practical Understanding of CPU Architecture

One of the primary objectives of this project is to provide a practical understanding of CPU architecture through the design, simulation, and implementation of an 8-bit processor. The CPU is the core component of every computing system, responsible for executing instructions, processing data, and controlling system operations. By designing an 8-bit CPU using Logisim Evolution and implementing it on an FPGA, students gain practical knowledge of how a processor performs the instruction cycle, including fetch, decode, and execute operations.

The project enables students to understand the interaction between essential CPU components such as the Arithmetic Logic Unit (ALU), registers, program counter, instruction register, memory unit, and control unit. Studying how these modules communicate and work together provides a comprehensive understanding of processor organization and instruction execution. Through simulation and hardware verification, students bridge the gap between theoretical concepts and practical implementation while developing a deeper appreciation of computer architecture.

1.1.2 Hands-On Experience with Digital System Design

Another important objective of this project is to provide hands-on experience in digital system design and hardware implementation. The CPU is developed by designing individual modules using Logisim Evolution**,** where each component is simulated, tested, and verified before being integrated into the complete processor. This modular approach helps students understand the functionality of each hardware block and its contribution to the overall system.

After successful simulation, the processor is implemented on an FPGA using Verilog, enabling students to experience the complete digital design flow from schematic development to hardware realization. During this process, students gain practical skills in circuit design, hardware verification, debugging, and system integration. These experiences strengthen problem-solving abilities and prepare students for advanced work in digital electronics, embedded systems, and VLSI design.

1.1.3 Educational Resource Development

A significant objective of the Anviksha – 8-bit CPU project is to develop a comprehensive educational resource that documents the complete process of CPU design, simulation, and FPGA implementation. The project includes detailed circuit diagrams, module descriptions, simulation results, and implementation procedures that help explain the operation of an 8-bit processor in a clear and systematic manner.

The developed resources can serve as valuable learning material for students, educators, and researchers interested in digital logic and processor design. By presenting both the theoretical concepts and practical implementation techniques, the project promotes active learning and encourages further exploration in computer architecture, FPGA-based system design, embedded systems, and VLSI engineering. It also provides a strong foundation for future enhancements, including expanded instruction sets, improved processor performance, and advanced architectural features.

1.2 Literature Survey

Sl. No.	Author(s) & Year	Title / Source	Methodology / Focus Area	Key Findings / Contributions	Limitations / Research Gap
1	Bhattacharya et al. (2025)	Exploring Microcode CPU Instructions with RISC 8-Bit Breadboard Computer	Microcoded control unit design using RISC architecture on breadboard	Demonstrates practical CPU design and instruction execution using microcode; useful for educational purposes	Limited scalability; lacks performance optimization and advanced instruction handling
2	Sharma et al. (2024)	CPU 101: Design of a Simple Microprocessor from First Principles	Design of CPU from basic digital logic and fundamental principles	Clearly explains datapath and control unit design from scratch	No hardware implementation; lacks real-time validation
3	Jain (2022)	RTL Design of CISC CPU IP Core	RTL-level HDL design of CISC processor	Efficient execution of complex instructions using CISC architecture	High complexity; not suitable for simple or beginner-level CPU design
4	Pyasi & Singh (2025)	Design and HDL Implementation of a Pipelined 8-bit Microprocessor	Pipeline-based processor design using HDL	Improves performance through pipelining techniques	Increased hardware complexity; difficult for breadboard implementation
5	Zhang (2024)	Design and Implementation of Digital Integrated Circuit Based CPU	Integrated circuit-based CPU design approach	Compact and efficient CPU implementation using IC technology	Focuses on IC-level design; not suitable for discrete component implementation
6	Saramud et al. (2024)	Selection of CPU architecture for control unit implementation in assembler	Comparative study of RISC and CISC architectures	Highlights importance of architecture selection for control unit design	Lacks practical hardware implementation; mostly theoretical analysis
7	Reddy et al. (2024)	Performance Comparison of 4, 8 and 16-bit ALU in FPGA Boards	FPGA-based ALU design and comparison study	Shows trade-offs between performance and hardware resource utilization	Limited to ALU only; does not cover complete CPU design

8	Gazziro et al. (2024)	Design and Evaluation of Open-Source Soft-Core Processors	Analysis and evaluation of open-source CPU architectures	Provides flexibility and insights into customizable processor design	Requires FPGA/tools; not beginner-friendly or hardware-level implementation
9	Mane et al. (2025)	FPGA Implementation of a Resource-Optimized 8-Bit Microcontroller	FPGA-based optimized microcontroller design	Efficient resource usage and suitable for educational applications	Not implemented using discrete components like breadboard systems

Over recent years, numerous studies have focused on the design and implementation of microprocessors, particularly 8-bit CPUs for educational and embedded applications. Bhattacharya et al. [1] demonstrated a microcoded RISC-based 8-bit breadboard computer, highlighting practical instruction execution but with limited scalability. Sharma et al. [2] explained CPU design from first principles, providing strong theoretical understanding without hardware validation. Jain [3] presented a CISC-based RTL CPU design, achieving efficient instruction handling but increasing complexity.

Further, Pyasi and Singh [4] introduced a pipelined 8-bit processor using HDL, improving performance but making hardware implementation difficult. Zhang [5] focused on integrated circuit-based CPU design, ensuring compactness but not suitable for discrete implementations. Saramud et al. [6] compared CPU architectures and emphasized the importance of selecting appropriate design approaches, though lacking practical validation.

Reddy et al. [7] analyzed ALU performance across different bit sizes using FPGA, showing efficiency trade-offs but not covering full CPU systems. Gazziro et al. [8] explored open-source soft-core processors, offering flexibility but requiring advanced tools. Mane et al. [9] implemented a resource-optimized 8-bit microcontroller on FPGA, achieving efficiency but lacking physical hardware realization.

From the literature, it is evident that most existing works focus on HDL-based processor design, FPGA implementation, processor optimization, or theoretical CPU architectures. While these approaches provide valuable insights into modern processor design, they often emphasize specific modules or advanced techniques rather than a complete educational processor. Therefore, this project aims to design, simulate, and implement an 8-bit CPU using Logisim Evolution**,** Verilog, and FPGA, providing a practical and systematic approach to understanding computer architecture and digital system design.

1.2.1 Summary

This literature survey highlights the methodologies and insights from various studies that guided the design and implementation of the Anviksha 8-bit CPU. The reviewed works emphasized modular CPU architecture, efficient Arithmetic Logic Unit (ALU) design, instruction execution, register organization, and control unit implementation. They also

demonstrated the importance of simulation and hardware realization using tools such as Logisim Evolution**,** Verilog**,** and FPGA platforms for developing reliable digital systems.

The survey further explored different CPU architectures, HDL-based processor design, FPGA implementation techniques, and processor performance optimization. These studies provided valuable guidance for designing an educational 8-bit CPU by adopting a modular design approach and validating each hardware module through simulation before FPGA implementation. The knowledge gained from the literature formed the foundation for developing a processor that is functionally reliable, scalable, and suitable for learning the principles of computer architecture, digital logic design, and hardware implementation.

CHAPTER 2

FUNDAMENTALS

A strong understanding of digital system design and computer architecture is essential for developing a functional processor. This chapter introduces the fundamental concepts required for the design and implementation of an 8-bit CPU. It covers the principles of digital logic, combinational and sequential circuits, processor architecture, FPGA technology, and simulation tools. These concepts provide the theoretical foundation for understanding the CPU design, instruction execution, and hardware implementation presented in the later chapters.

2.1 Fundamentals of Digital Logic

Digital logic is the foundation of modern digital systems, including microprocessors, microcontrollers, embedded systems, and computers. Digital circuits operate using binary values, where logic '1' represents a HIGH voltage level and logic '0' represents a LOW voltage level. These binary values are processed using logic gates that perform arithmetic and logical operations.

In the proposed 8-bit CPU, digital logic is used to design the Arithmetic Logic Unit (ALU), registers, multiplexers, control unit, program counter, instruction decoder, and memory interface. The entire processor is built by combining simple logic gates into larger functional modules.

2.1.1 Combining Logic Gates to Create Complex Digital Circuits

Complex digital circuits are formed by combining basic logic gates such as AND, OR, NOT, and XOR. While each logic gate performs a simple Boolean operation, their combination enables the implementation of sophisticated digital systems, including processors.

In the proposed 8-bit CPU, logic gates are integrated to design functional modules such as the Arithmetic Logic Unit (ALU), multiplexers, decoders, registers, program counter, and control unit. These modules work together to execute arithmetic, logical, and data transfer operations.

For example, an 8-bit adder is constructed by cascading multiple full-adder circuits, allowing arithmetic operations on 8-bit binary numbers. Similarly, multiplexers are used to select data from multiple sources based on control signals, while decoders generate unique output signals for instruction decoding and memory selection.

The integration of these digital components forms the complete processor architecture, enabling efficient instruction execution and data processing.

2.1.2 Combinational and Sequential Logic

Digital systems are broadly classified into combinational and sequential circuits.

Combinational circuits generate outputs solely based on the present inputs. Examples include adders, subtractions, multiplexers, decoders, and logic units. These circuits form the computational components of the CPU.

Sequential circuits generate outputs based on both present inputs and previous states. They use storage elements such as flip-flops and registers to store data. Components like the Program Counter (PC), Instruction Register (IR), and General-Purpose Registers are sequential circuits that enable instruction execution and data storage.

2.2 Fundamentals of Computer Architecture

Computer architecture defines the structure and organisation of a processor. An 8-bit CPU processes 8-bit data and executes instructions sequentially through the Fetch–Decode–Execute cycle.

The processor designed in this project consists of the following major components:

- Arithmetic Logic Unit (ALU)
- Program Counter (PC)
- Instruction Register (IR)
- General-Purpose Registers
- Control Unit
- RAM and ROM
- Multiplexers and Data Bus

The Control Unit generates control signals that coordinate data movement and processor operations, while the ALU performs arithmetic and logical computations. The Program Counter stores the address of the next instruction, and the Instruction Register holds the currently executing instruction.

2.3 Overview of 8-Bit CPU Architecture

A Central Processing Unit (CPU) is the primary component responsible for executing program instructions and controlling the operation of a digital system. It performs arithmetic, logical, data transfer, and control operations through the coordinated functioning of multiple hardware modules.

The proposed 8-bit CPU processes 8-bit data and executes instructions following the Fetch–Decode–Execute cycle. The processor consists of several major components, including:

- Arithmetic Logic Unit (ALU)
- Program Counter (PC)
- Instruction Register (IR)
- General-Purpose Registers
- Control Unit
- RAM and ROM
- Data Bus and Control Bus

Each component performs a specific function, and together they enable the processor to execute instructions efficiently.

2.3.1 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the computational core of the processor. It performs arithmetic and logical operations on 8-bit binary data, making it one of the most important components of the CPU.

The ALU in the proposed processor supports operations such as:

- Addition
- Subtraction
- AND
- OR
- XOR
- NOT
- Data comparison (if implemented)

These operations are performed using combinations of logic gates, multiplexers, and arithmetic circuits. The ALU receives operands from the register file, performs the selected operation based on the control signals, and stores the result in the destination register.

The performance of the processor largely depends on the efficiency of the ALU, as every arithmetic and logical instruction passes through this unit.

2.3.2 Registers

Registers are high-speed storage elements located inside the CPU. They temporarily store instructions, operands, addresses, and intermediate computation results during program execution.

The proposed 8-bit CPU contains several registers, including:

- General-Purpose Registers for storing data.
- Program Counter (PC) to hold the address of the next instruction.
- Instruction Register (IR) to store the currently executing instruction.
- Accumulator (if implemented) to store ALU results.

Registers are implemented using sequential logic circuits and operate synchronously with the system clock. They provide rapid access to data, improving the overall performance of the processor by reducing memory access delays.

2.3.3 Control Unit

The Control Unit (CU) coordinates the operation of all components within the processor. It manages the flow of data between the ALU, registers, memory, and input/output units by generating appropriate control signals.

During instruction execution, the Control Unit performs the following functions:

- Fetches the instruction from memory.
- Decodes the instruction opcode.
- Generates the required control signals.
- Controls data movement between registers and memory.
- Initiates ALU operations.
- Updates the Program Counter.

The processor follows the Fetch–Decode–Execute instruction cycle. During the Fetch stage, the instruction is read from memory. In the Decode stage, the Control Unit interprets the instruction opcode and determines the required operation. Finally, during the Execute stage, the ALU or other processor modules perform the specified operation, and the result is stored appropriately.

The Control Unit ensures that all processor modules operate in the correct sequence, enabling reliable and efficient execution of instructions.

2.3.4 Program Counter (PC)

The Program Counter (PC) is a special-purpose register that stores the memory address of the next instruction to be executed by the CPU. During the instruction execution process, the Program Counter ensures that instructions are fetched from memory in the correct sequence.

In the proposed 8-bit CPU**,** the Program Counter is updated after every instruction execution. During the fetch stage, the address stored in the Program Counter is sent to the instruction memory to retrieve the next instruction. After fetching the instruction, the Program Counter is incremented so that it points to the next instruction in memory.

The Program Counter plays a crucial role in maintaining the correct execution flow of a program. It also supports branching and jump instructions by allowing the processor to load a new address instead of simply incrementing the current one. Proper functioning of the Program Counter is essential for sequential instruction execution and overall processor control.

2.3.5 Instruction Register (IR)

The Instruction Register (IR**)** is a dedicated register that temporarily stores the instruction fetched from memory before it is decoded and executed. It acts as an interface between the instruction memory and the Control Unit.

In the proposed 8-bit CPU, once an instruction is fetched from memory using the address provided by the Program Counter, it is loaded into the Instruction Register. The Control Unit then reads the instruction stored in the IR, decodes the opcode, and generates the necessary control signals required to perform the specified operation.

The Instruction Register ensures that the current instruction remains available throughout the execution cycle, allowing the processor to complete all required operations before fetching the next instruction. This contributes to stable and reliable instruction execution.

2.3.6 RAM and ROM

Memory is an essential component of every processor as it stores both instructions and data required during program execution. The proposed 8-bit CPU uses Read Only Memory (ROM) and Random Access Memory (RAM) for this purpose.

Read Only Memory (ROM) stores the program instructions permanently. The CPU fetches instructions from ROM during the execution process. Since the contents of ROM do not change during normal operation, it provides reliable storage for the instruction set used by the processor.

Random Access Memory (RAM**)** is used to store temporary data generated during program execution. It allows both read and write operations, enabling the processor to store intermediate results, variables, and user data. The ALU accesses RAM whenever data needs to be processed or updated.

The combination of ROM for instruction storage and RAM for data storage enables efficient execution of programs and smooth data transfer within the CPU architecture.

2.3.7 Instruction Execution Cycle

The Instruction Execution Cycle is the sequence of operations performed by the CPU to execute a program instruction. Every instruction passes through three fundamental stages: Fetch**,** Decode**,** and Execute**.**

Fetch: During the fetch stage, the Program Counter provides the memory address of the next instruction. The CPU accesses the instruction memory (ROM) and retrieves the instruction, which is then stored in the Instruction Register. After the instruction is fetched, the Program Counter is updated to point to the next instruction.

Decode: In the decode stage, the Control Unit interprets the instruction stored in the Instruction Register. It identifies the operation to be performed by examining the instruction opcode and generates the required control signals. These control signals coordinate the operation of the ALU, registers, and memory.

Execute: During the execute stage, the processor performs the required operation specified by the instruction. Arithmetic and logical operations are carried out by the ALU, while data transfer instructions move data between registers and memory. After execution, the result is stored in the appropriate destination register or memory location, and the processor proceeds to fetch the next instruction.

The continuous repetition of the Fetch–Decode–Execute cycle enables the CPU to execute programs efficiently and accurately, making it the fundamental operating principle of every processor.

2.4 Introduction to Logisim Evolution

Logisim Evolution is an open-source digital logic design and simulation software that provides a graphical environment for designing, testing, and analysing digital circuits. It enables users to create digital systems by placing logic gates, registers, multiplexers, memory blocks, counters, and other components on a workspace and connecting those using wires.

Unlike text-based hardware description languages, Logisim Evolution offers a visual approach to digital circuit design, making it an effective platform for understanding processor architecture and digital system behaviour.

In this project, Logisim Evolution was used to design, simulate, and verify the complete 8-bit CPU architecture**.** Each functional module was individually developed and tested before integrating them into the complete processor. The simulation environment helped validate

instruction execution, arithmetic operations, register transfers, and control logic prior to FPGA implementation.

2.3.1 Key Features of Logisim Evolution

Graphical User Interface
Logisim Evolution provides a user-friendly graphical interface where digital circuits can be designed using a drag-and-drop approach. Users can easily place components, connect them using wires, and modify circuit configurations without writing hardware description code.

Circuit Simulation
The software provides real-time simulation of digital circuits. During simulation, users can observe signal transitions, monitor register contents, verify ALU operations, and analyse instruction execution. This feature greatly simplifies debugging and functional verification.

Extensive Component Library

Logisim Evolution includes a comprehensive library of digital components such as:

- Logic Gates
- Multiplexers
- Decoders
- Encoders
- Registers
- Counters
- ROM and RAM
- Arithmetic Components
- Flip-Flops
- Splitters and Tunnels

These components enable rapid development of complex digital systems without designing every circuit from basic gates.

Sub-Circuit Design
One of the major advantages of Logisim Evolution is its support for modular design through sub-circuits. Complex systems can be divided into smaller functional blocks such as the ALU, Register File, Control Unit, Program Counter, and Memory Unit.
This modular approach improves circuit readability, simplifies debugging, and allows individual modules to be reused in future processor designs.

2.5 Why Logisim Evolution

Logisim Evolution was selected for this project because it provides a simple yet powerful environment for designing and simulating digital systems. It allows complete processor architectures to be developed visually while providing real-time verification of circuit functionality.

For the proposed 8-bit CPU, Logisim Evolution offered several advantages:

- Easy graphical design of processor components.
- Real-time simulation and debugging.
- Modular circuit development using sub-circuits.
- Fast verification before hardware implementation.
- Educational platform for understanding CPU architecture.

The software significantly reduced development time by enabling errors to be detected during simulation before implementing the processor on hardware.

2.6 FPGA and Basys-3 Development Board

A Field Programmable Gate Array (FPGA) is a programmable integrated circuit that allows designers to implement custom digital hardware after manufacturing. Unlike conventional processors with fixed architectures, FPGAs can be configured to perform specific digital functions according to application requirements.

After completing the simulation in Logisim Evolution, the designed 8-bit CPU was implemented on the Basys-3 FPGA development board**,** which is based on the Xilinx Artix-7 FPGA**.**

The FPGA implementation validates the processor in actual hardware, allowing the designed CPU to execute instructions in real time. This bridges the gap between software simulation and practical hardware implementation while providing higher processing speed and flexibility.

2.6.1 Basys-3 FPGA Overview

The Basys-3 is an educational FPGA development board designed by Digilent and built around the Xilinx Artix-7 FPGA**.** It provides a complete hardware platform for implementing and testing digital systems.

The board includes several built-in peripherals such as:

- User LEDs
- Slide switches
- Push buttons
- Seven-segment displays
- Clock generator
- USB programming interface
- VGA interface
- PMOD expansion connectors

These peripherals enable users to interact directly with digital circuits and observe processor outputs during hardware execution.

Because of its reliability, affordability, and rich set of on-board peripherals, the Basys-3 board is widely used in engineering education, embedded system development, and digital design projects.

2.6.2 Advantages of FPGA Implementation

Implementing the designed processor on an FPGA provides several benefits over software simulation alone.

- Real-time execution of digital hardware.
- Faster operation through hardware implementation.
- Easy modification and reconfiguration of processor architecture.
- Efficient testing and debugging of processor functionality.
- Practical understanding of computer architecture and digital system design.
- Opportunity to validate the processor under actual hardware conditions.

FPGA implementation also enables future expansion of the processor by adding new instructions, peripherals, or memory modules without redesigning the hardware from scratch.

2.7 Why Basys-3 FPGA

The Basys-3 FPGA development board was selected for this project because it provides an ideal platform for implementing and testing custom processor architectures.

The board offers a balanced combination of processing capability, ease of use, and educational features. Its on-board switches, LEDs, push buttons, and seven-segment displays enable real-time interaction with the processor without requiring additional hardware.

The Artix-7 FPGA provides sufficient programmable logic resources to implement the complete 8-bit CPU, making it suitable for processor development and digital system experimentation. Its reconfigurable architecture allows future enhancements such as expanding the instruction set, increasing memory capacity, or integrating additional peripherals.

The successful implementation of the proposed 8-bit CPU on the Basys-3 FPGA demonstrates the practicality and reliability of the processor design beyond simulation, providing both functional verification and real-world hardware validation.